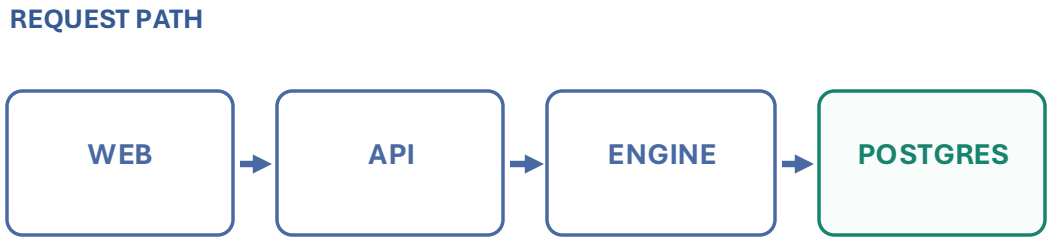
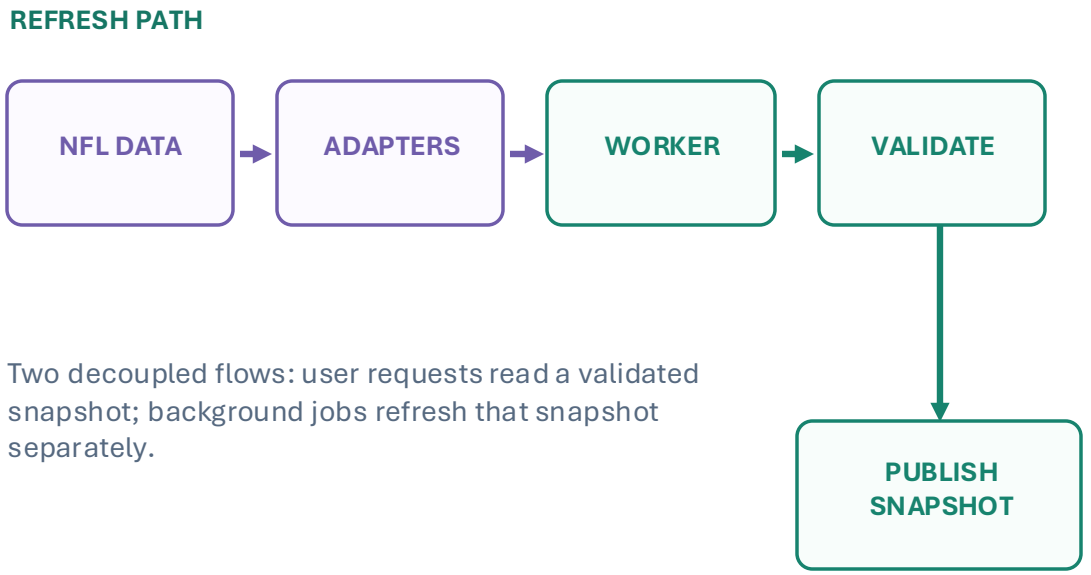
V1 ARCHITECTURE

Fantasy Football Recommender

A modular full-stack system that turns refreshed NFL data into valid, explainable lineup recommendations.





Two decoupled flows: user requests read a validated snapshot; background jobs refresh that snapshot separately.

MODULAR • EXPLAINABLE • SAFE

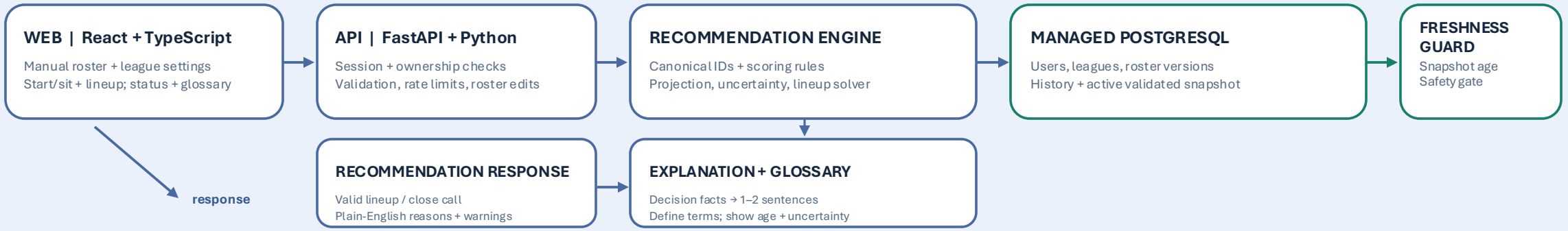
V1 ARCHITECTURE

Full system architecture

User requests consume a validated snapshot; scheduled jobs refresh and publish that snapshot independently.

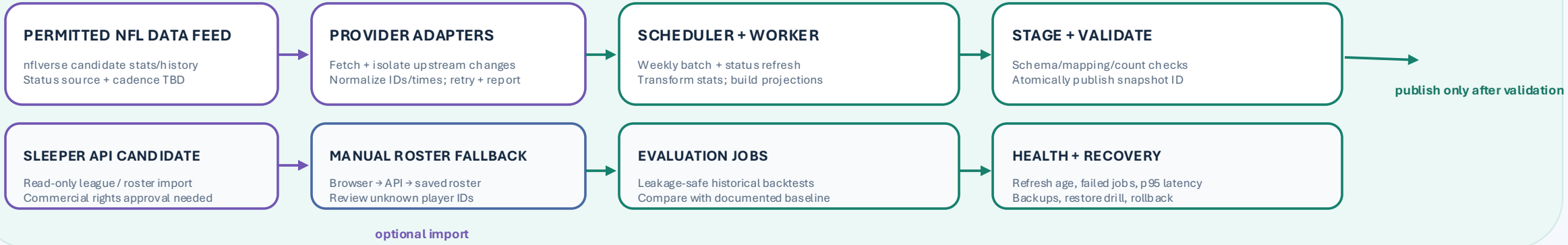
01 INTERACTIVE REQUEST | HTTPS / JSON

Requests read the last published snapshot. External provider fetches run separately.



02 SCHEDULED REFRESH | INDEPENDENT OF USER REQUESTS

External sources are candidates; terms, rights, field coverage, and freshness must be confirmed.



DESIGN GUARDRAILS

Import requires provider permission • Requests never wait for external feeds • Partial snapshots are never served • Waivers/trades are later modules

The stack in one view

Four core responsibilities, with data refresh intentionally separated from user traffic.

WEB

React + TypeScript

Manual roster and league settings
Start/sit and full-lineup views
Status, reasons, glossary

API

FastAPI + Python

Sessions and ownership checks
Validation and rate limiting
Roster edits and optional import

LOGIC

Recommendation Engine

Canonical player IDs
Projection + uncertainty
Eligibility-aware lineup solver

DATA

Managed PostgreSQL

Users, leagues, roster versions
Player/scoring/week history
Active validated snapshot ID

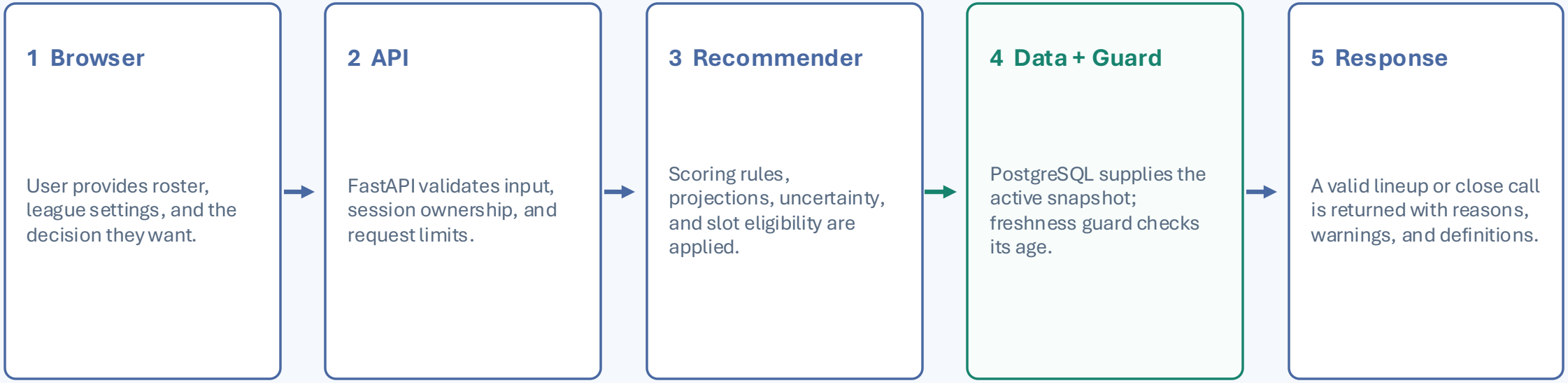
Design principle

Interactive requests never wait on an external provider. They read the last validated snapshot, so responsiveness and upstream reliability are decoupled.

CORE APPLICATION STACK

Interactive request flow

What happens when a user asks for a start/sit decision or a full lineup.



Freshness rule The UI exposes source age and uncertainty; unsafe injury-dependent picks can be paused instead of silently presented as current.

USER-FACING PATH

Inside the recommendation layer

The “smart” part is deterministic enough to test, explain, and backtest.

1 **Canonicalize the league**

Map players, scoring settings, week, positions, and roster slots into one internal representation.

2 **Build player estimates**

Use refreshed stats/history to produce projections plus uncertainty rather than a single opaque score.

3 **Solve the lineup**

Respect slot eligibility and optimize the full lineup; support close-call start/sit comparisons.

4 **Explain the decision**

Generate 1–2 sentence reasons from the same decision facts; define fantasy terms in a glossary.

Output contract

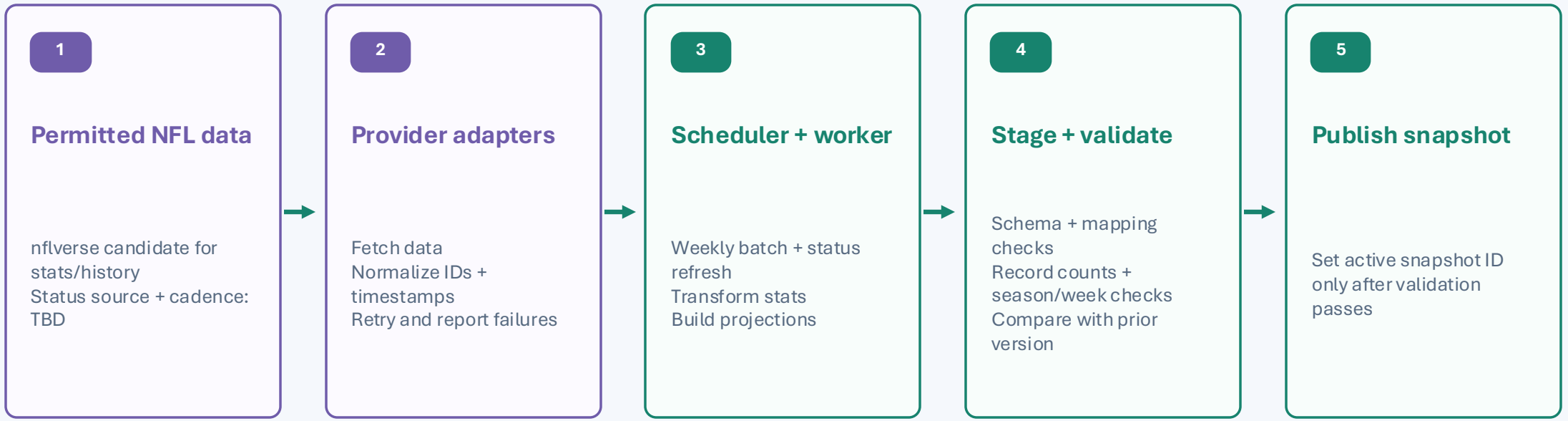
Recommendation	Who starts / which lineup is valid
Confidence	How close the alternatives are
Reasons	Facts that drove the decision
Warnings	Freshness or status limitations
Definitions	Plain-English glossary terms

Because explanations use the same decision facts, the user-facing reasoning stays tied to the actual recommendation logic.

RECOMMENDATION ENGINE

Scheduled refresh pipeline

External data is fetched, normalized, validated, then atomically published.



Optional Sleeper API import

Read-only league/roster import; commercial rights approval still required.

feeds adapters

Key guarantee

Partial or failed refreshes never become the active dataset.

BACKGROUND DATA PATH

Reliability, evaluation, and V1 guardrails

The architecture is deliberately conservative where stale or malformed data could create bad recommendations.

FALLBACK

Manual roster fallback

Browser → API → saved roster
Unknown player IDs are reviewed instead of guessed

QUALITY

Evaluation jobs

Leakage-safe historical backtests
Compare against a documented baseline

OPS

Health + recovery

Track refresh age, failed jobs, and p95 latency
Backups, restore drills, and rollback

V1 boundaries

- **No request-time provider dependency** — User traffic reads validated stored data.
- **No partial snapshots** — Publish only after all validation checks pass.
- **No live in-game updates** — Refresh cadence is batch/status oriented.
- **No premature scaling** — Redis, queues, and multi-instance API follow measured need.
- **Later modules** — Waivers and trades are explicit follow-on scope.

OPERABILITY + SCOPE

Why the architecture is shaped this way

A small set of design choices keeps the system understandable, testable, and extensible.

1. **Fast user experience** — Requests use a prebuilt snapshot instead of waiting on third-party APIs.
2. **Safer recommendations** — Freshness and validation are first-class system concerns, not UI afterthoughts.
3. **Explainable behavior** — The same facts drive both the recommendation and the explanation.
4. **Replaceable providers** — Adapters isolate external API changes from the application core.
5. **Measured scaling** — The V1 stays simple; more infrastructure appears only when traffic justifies it.

Result: a V1 that is simple enough to build now, but already has clear seams for new data providers, waiver/trade modules, and higher scale later.